Python Activity Based Learning

Selected Projects - Reports, Code, Test Cases and Flowcharts

Contents

1. Electricity Bill 2

2. Employee Payroll 6

3. Retail Bill 10

4. Examination Result 14

5. Attendance Recovery 18

6. ATM Transaction 22

7. Inventory Reconciliation 26

8. Water Tank Simulation 30

9. Number Diamond Pattern 34

10. Password Validator 38

Each project contains the report sections and a program screenshot.

Electricity Bill

Problem Statement

Calculate a household electricity bill from the number of units consumed. Use three unit slabs and add a fixed charge.

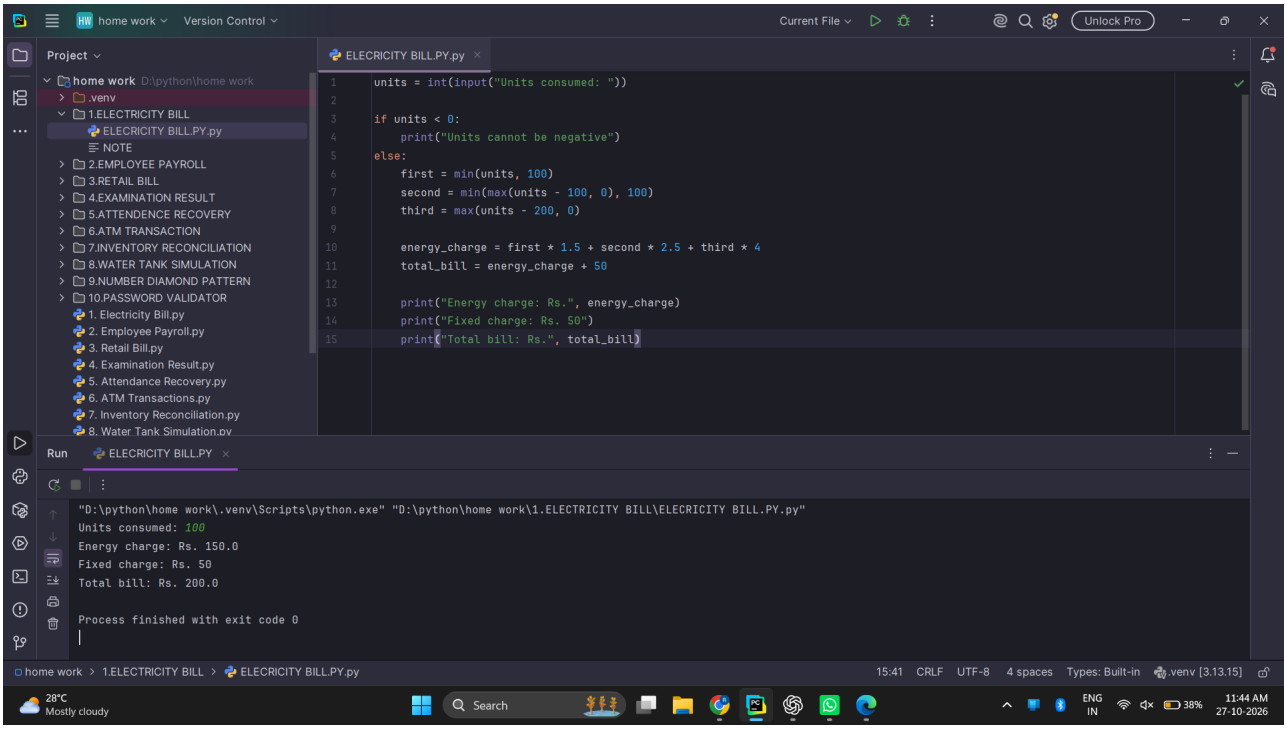
Objectives

To use input, arithmetic operations, if-else statements, and the min and max functions.

Assumptions

The first 100 units cost Rs. 1.50 per unit, the next 100 cost Rs. 2.50 per unit, and units above 200 cost Rs. 4 per unit. The fixed charge is Rs. 50. These are sample rates.

Program Screenshot



Electricity Bill - Algorithm and Pseudocode

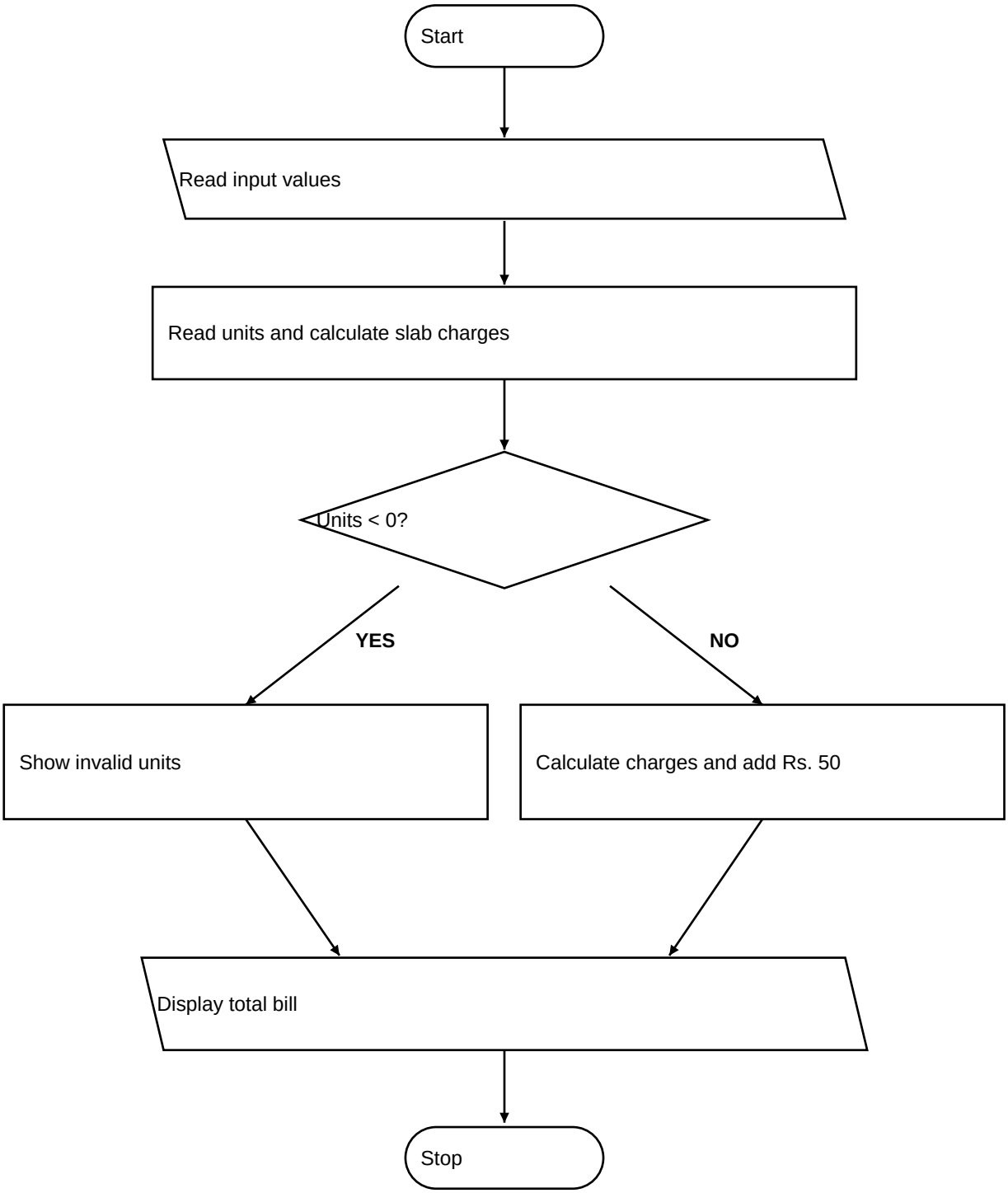
Algorithm

1. Start the program.
2. Read the number of units.
3. If units are negative, show an error message.
4. Otherwise, calculate the charge for each slab.
5. Add Rs. 50 to the energy charge.
6. Display the energy charge and total bill.

Pseudocode

```
START
INPUT units
IF units < 0
    DISPLAY "Units cannot be negative"
ELSE
    first = minimum(units, 100)
    second = minimum(maximum(units - 100, 0), 100)
    third = maximum(units - 200, 0)
    energy = first * 1.5 + second * 2.5 + third * 4
    total = energy + 50
    DISPLAY energy, fixed charge, total
END IF
STOP
```

Electricity Bill - Flowchart



For repeated items / days / transactions, repeat the input and process steps as required.

Electricity Bill - Source Code and Test Cases

Python Source Code

```
units = int(input("Units consumed: "))

if units < 0:
    print("Units cannot be negative")
else:
    first = min(units, 100)
    second = min(max(units - 100, 0), 100)
    third = max(units - 200, 0)

    energy_charge = first * 1.5 + second * 2.5 + third * 4
    total_bill = energy_charge + 50

    print("Energy charge: Rs.", energy_charge)
    print("Fixed charge: Rs. 50")
    print("Total bill: Rs.", total_bill)
```

Test Cases and Program Output

Input: 100 units
Expected output: Energy Rs. 150; fixed Rs. 50; total Rs. 200

Input: -5 units
Expected output: Units cannot be negative

Conclusion

The program calculates a bill from unit slabs and displays the total amount.

Employee Payroll

Problem Statement

Calculate an employee's gross pay, deduction, and net pay using basic pay, allowances, and overtime.

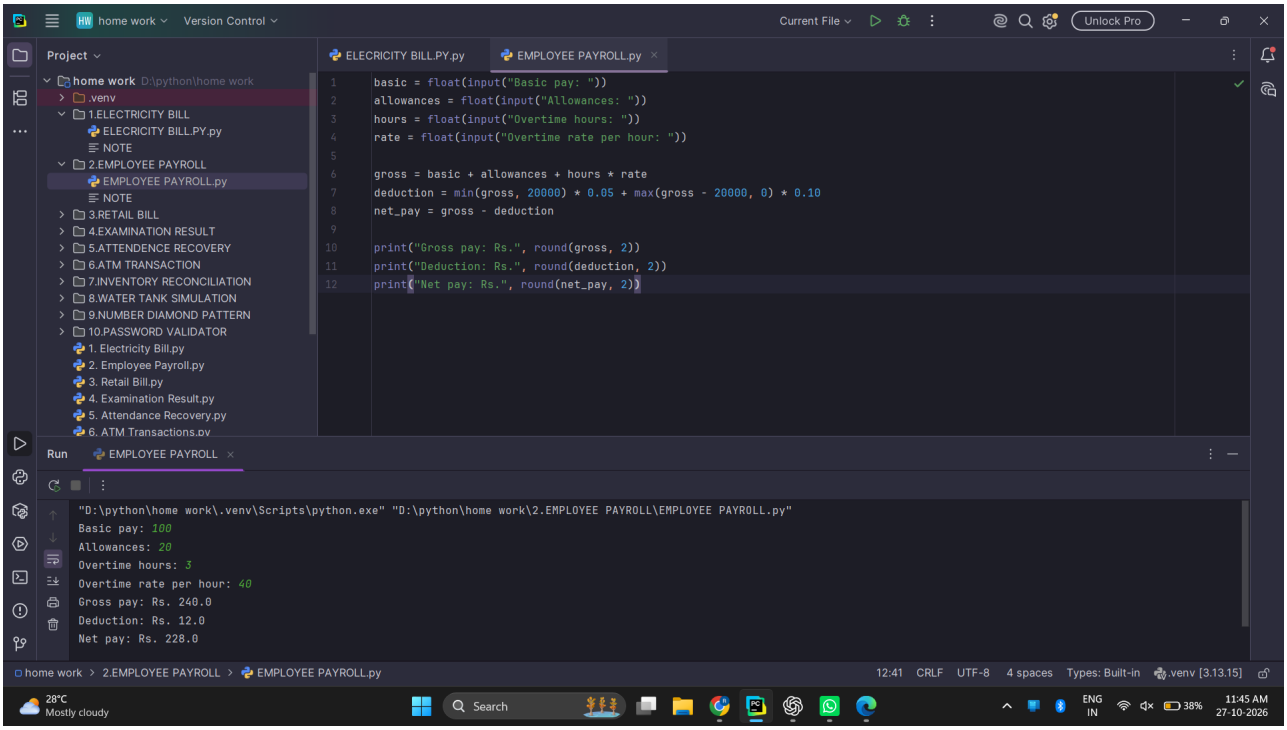
Objectives

To practice input, variables, arithmetic, and progressive deductions.

Assumptions

Five percent is deducted from the first Rs. 20,000 of gross pay and 10% from any amount above Rs. 20,000. The rates are for this exercise.

Program Screenshot



Employee Payroll - Algorithm and Pseudocode

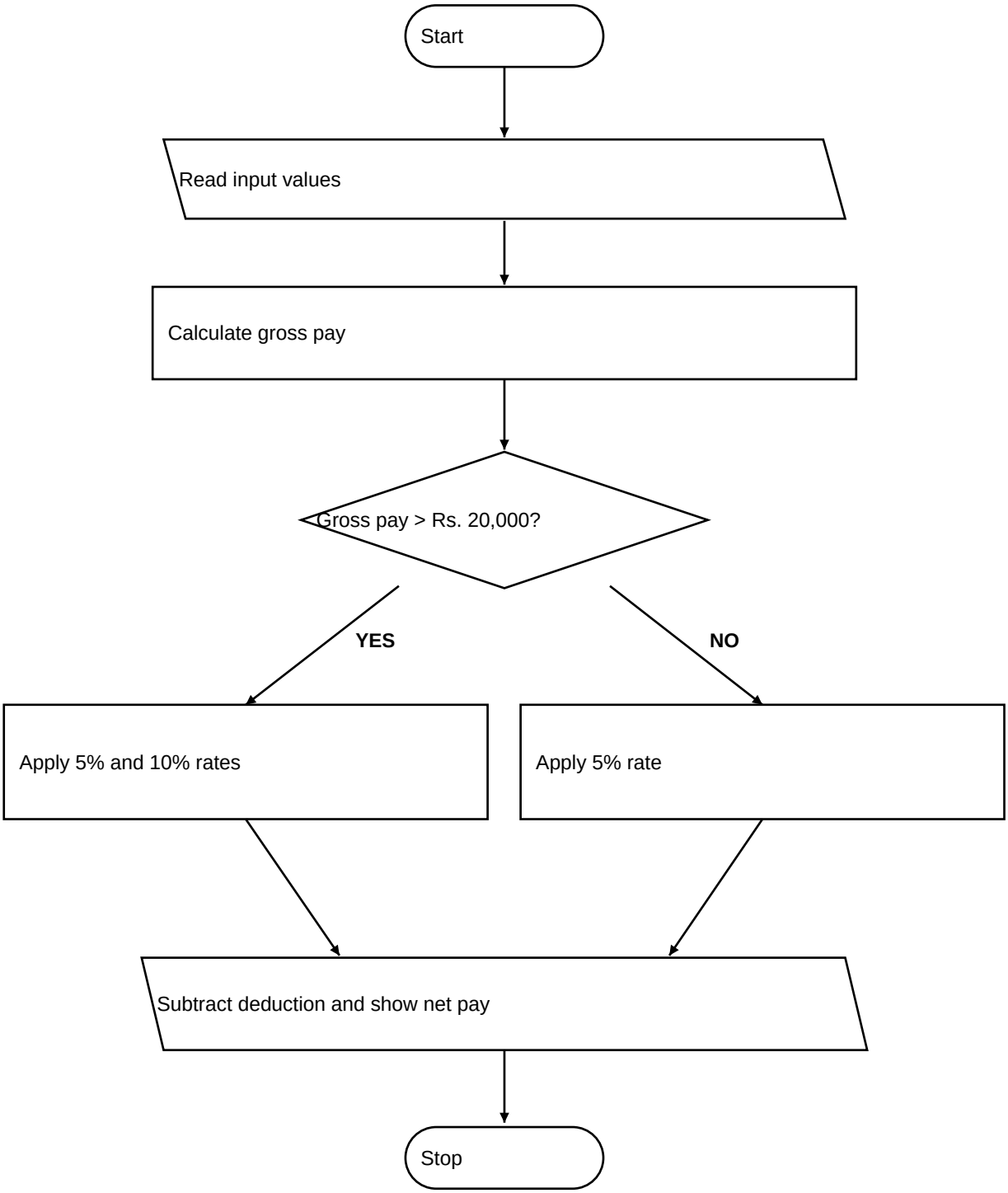
Algorithm

1. Read basic pay, allowances, overtime hours, and overtime rate.
2. Calculate overtime pay.
3. Add basic pay, allowances, and overtime pay.
4. Calculate the deduction for both pay ranges.
5. Subtract the deduction from gross pay.
6. Display gross pay, deduction, and net pay.

Pseudocode

```
START
INPUT basic, allowances, hours, rate
overtime_pay = hours * rate
gross = basic + allowances + overtime_pay
first_part = minimum(gross, 20000)
extra_part = maximum(gross - 20000, 0)
deduction = first_part * 0.05 + extra_part * 0.10
net_pay = gross - deduction
DISPLAY gross, deduction, net_pay
STOP
```

Employee Payroll - Flowchart



For repeated items / days / transactions, repeat the input and process steps as required.

Employee Payroll - Source Code and Test Cases

Python Source Code

```
basic = float(input("Basic pay: "))
allowances = float(input("Allowances: "))
hours = float(input("Overtime hours: "))
rate = float(input("Overtime rate per hour: "))

gross = basic + allowances + hours * rate
deduction = min(gross, 20000) * 0.05 + max(gross - 20000, 0) * 0.10
net_pay = gross - deduction

print("Gross pay: Rs.", round(gross, 2))
print("Deduction: Rs.", round(deduction, 2))
print("Net pay: Rs.", round(net_pay, 2))
```

Test Cases and Program Output

Input: Basic 100, allowance 20, 3 hours at 40
Expected output: Gross Rs. 240; deduction Rs. 12; net Rs. 228

Input: All inputs zero
Expected output: Gross Rs. 0; deduction Rs. 0; net Rs. 0

Conclusion

The program adds salary components, calculates the deduction, and prints the take-home pay.

Retail Bill

Problem Statement

Prepare a bill for several products using their prices and quantities. Apply a discount when the subtotal reaches the given limit, then add tax.

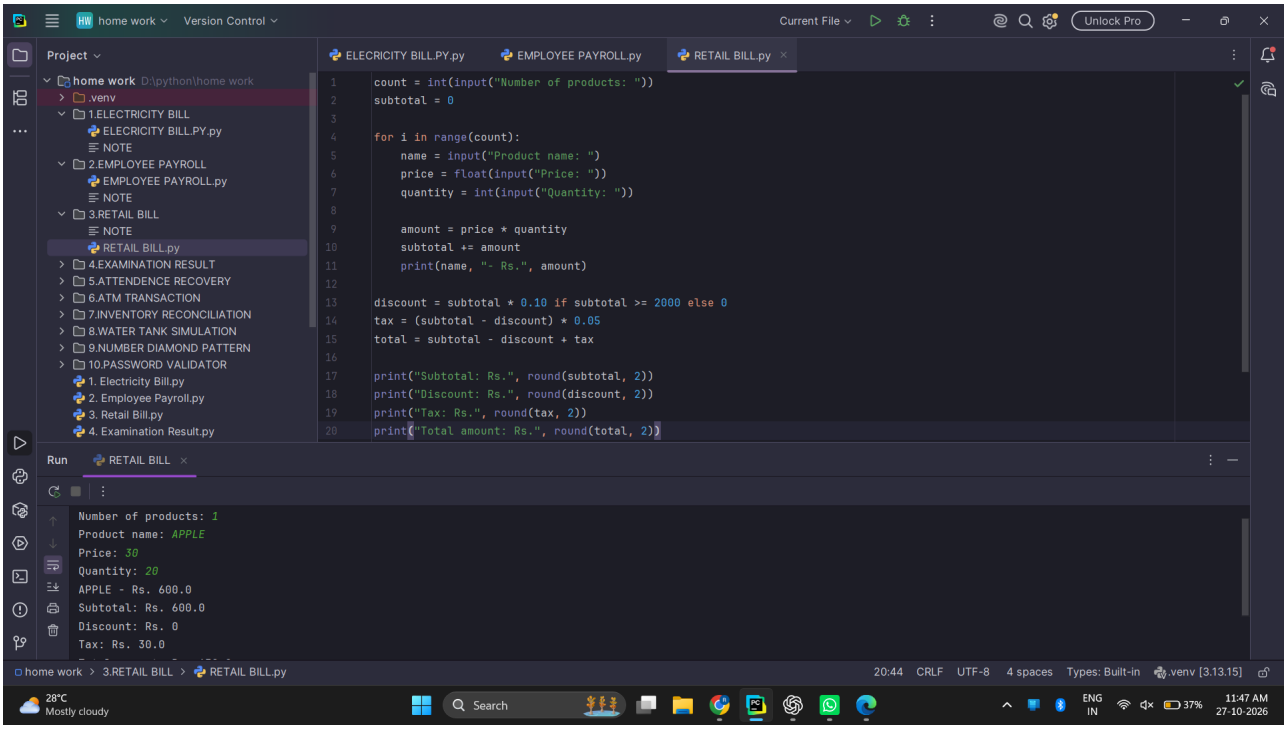
Objectives

To use a loop, input, multiplication, addition, and a conditional expression.

Assumptions

A 10% discount applies when the subtotal is at least Rs. 2,000. Tax is 5% of the amount after discount. Rates are examples for this activity.

Program Screenshot



Retail Bill - Algorithm and Pseudocode

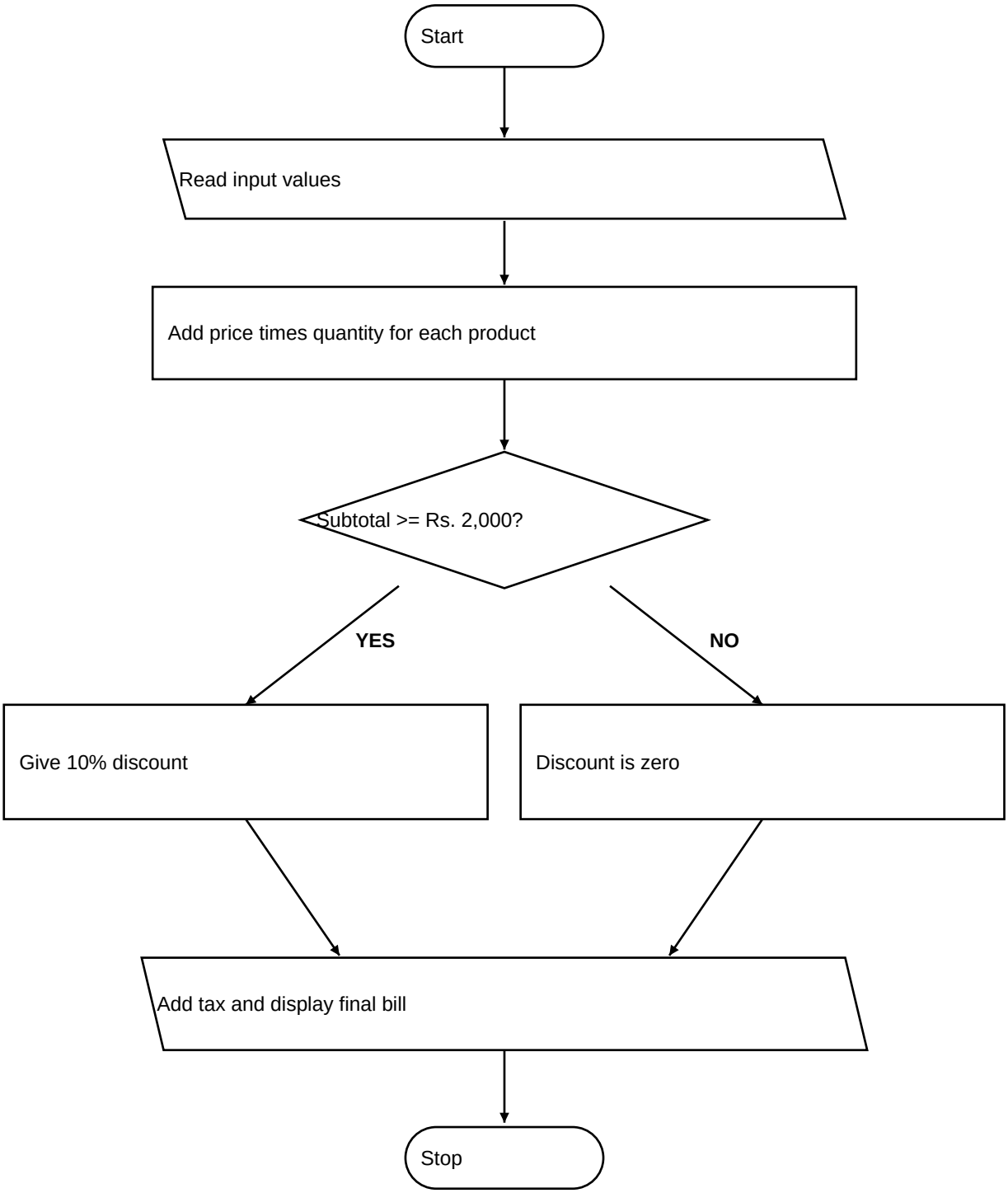
Algorithm

1. Read the number of products.
2. For each product, read name, price, and quantity.
3. Calculate the item amount and add it to subtotal.
4. If subtotal is Rs. 2,000 or more, calculate a 10% discount.
5. Calculate 5% tax on the discounted amount.
6. Display subtotal, discount, tax, and total.

Pseudocode

```
START
INPUT product_count
subtotal = 0
FOR each product
    INPUT name, price, quantity
    amount = price * quantity
    subtotal = subtotal + amount
    DISPLAY name, amount
END FOR
IF subtotal >= 2000, discount = subtotal * 0.10
ELSE discount = 0
tax = (subtotal - discount) * 0.05
total = subtotal - discount + tax
DISPLAY subtotal, discount, tax, total
STOP
```

Retail Bill - Flowchart



For repeated items / days / transactions, repeat the input and process steps as required.

Retail Bill - Source Code and Test Cases

Python Source Code

```
count = int(input("Number of products: "))
subtotal = 0

for i in range(count):
    name = input("Product name: ")
    price = float(input("Price: "))
    quantity = int(input("Quantity: "))

    amount = price * quantity
    subtotal += amount
    print(name, "- Rs.", amount)

discount = subtotal * 0.10 if subtotal >= 2000 else 0
tax = (subtotal - discount) * 0.05
total = subtotal - discount + tax

print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("Tax: Rs.", round(tax, 2))
print("Total amount: Rs.", round(total, 2))
```

Test Cases and Program Output

Input: 1 product: price 30, quantity 20
Expected output: Subtotal Rs. 600; discount Rs. 0; tax Rs. 30; total Rs. 630

Input: Subtotal Rs. 2,000
Expected output: Discount Rs. 200 before tax

Conclusion

The program adds the item amounts and applies the discount and tax rules to make a final bill.

Examination Result

Problem Statement

Read marks for three subjects, check that they are valid, and display the student's result class.

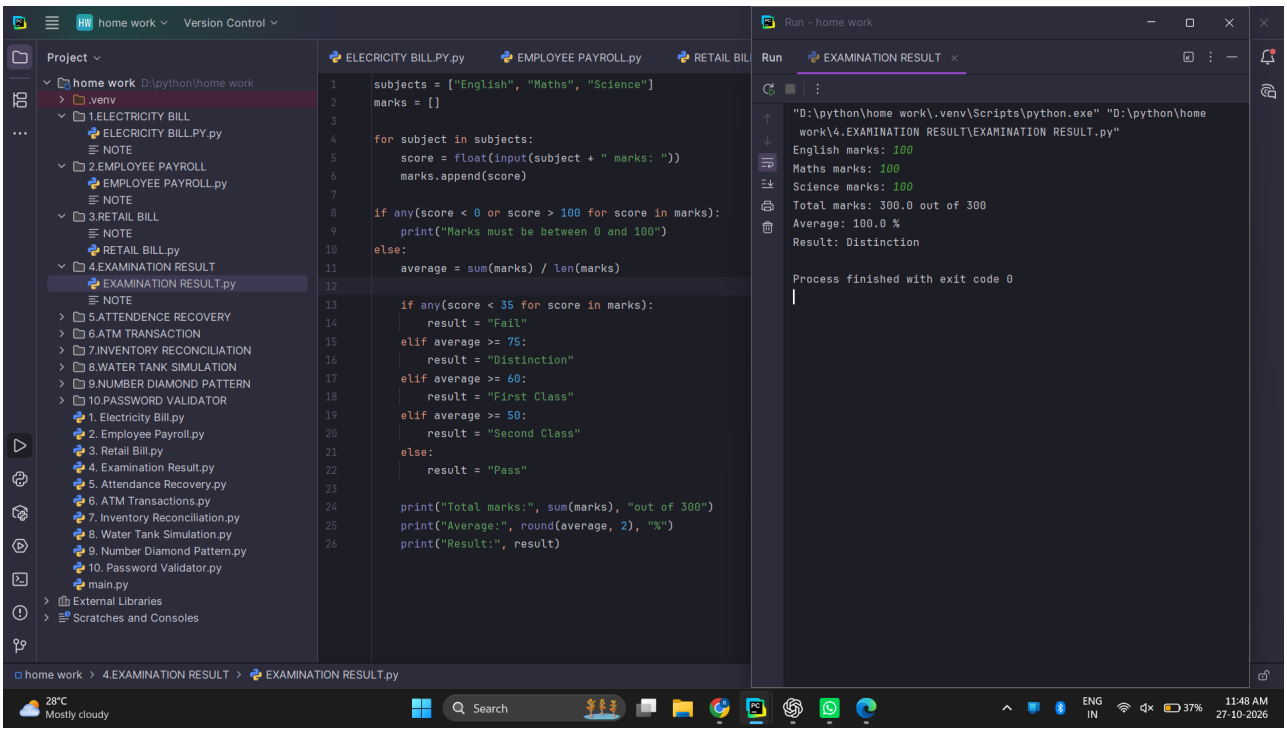
Objectives

To use lists, loops, conditions, sum, average, and pass criteria.

Assumptions

Marks are out of 100. A score below 35 in any subject is a fail. The average boundaries in the code decide the result class.

Program Screenshot



Examination Result - Algorithm and Pseudocode

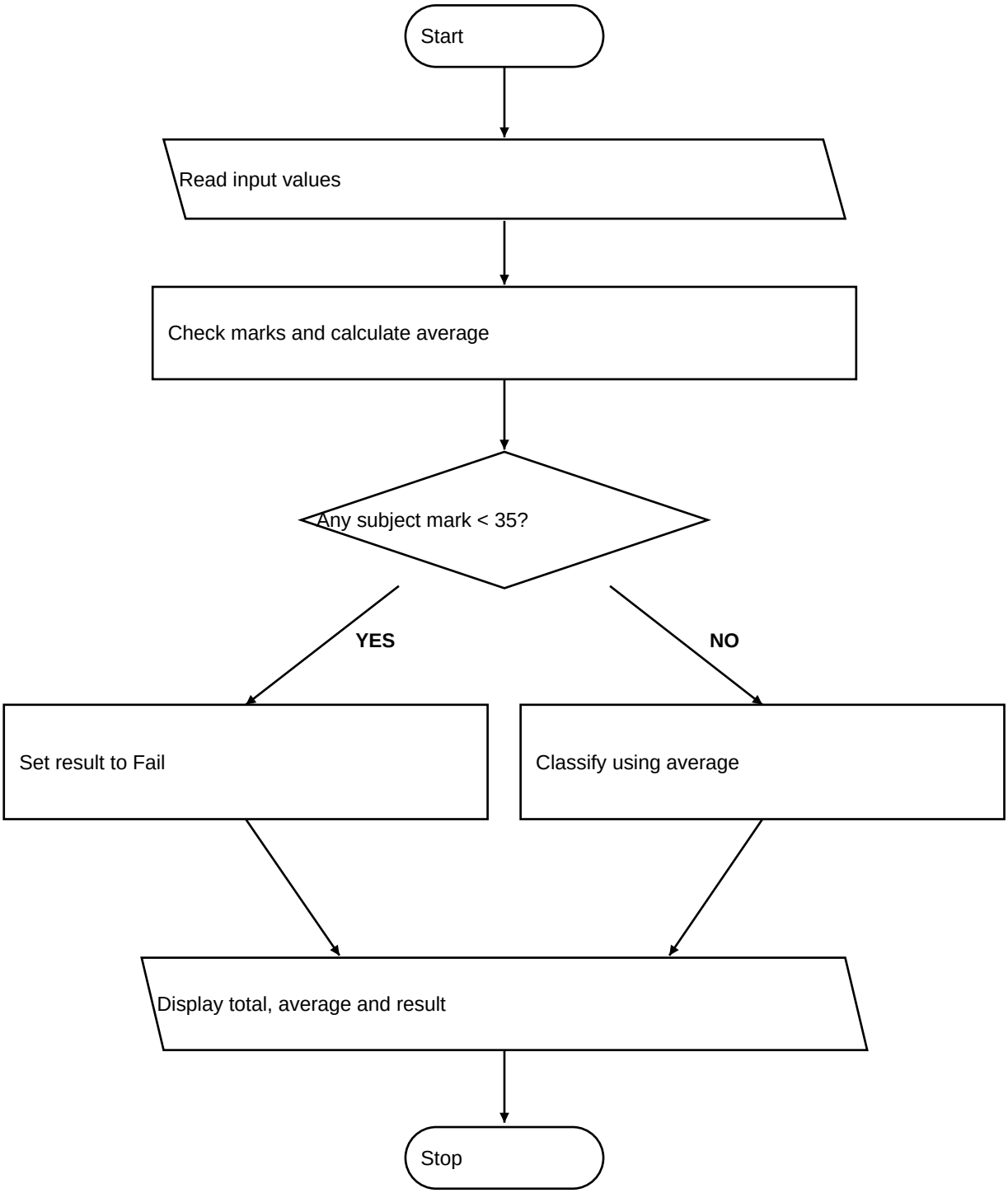
Algorithm

1. Read marks for English, Maths, and Science.
2. Check that every mark is between 0 and 100.
3. If a mark is invalid, display an error.
4. Otherwise calculate total and average.
5. If any mark is below 35, set result to Fail.
6. Otherwise choose the class using the average.
7. Display total, average, and result.

Pseudocode

```
START
subjects = English, Maths, Science
INPUT marks for each subject
IF any mark < 0 OR any mark > 100
    DISPLAY invalid marks
ELSE
    average = total marks / 3
    IF any mark < 35, result = Fail
    ELSE IF average >= 75, result = Distinction
    ELSE IF average >= 60, result = First Class
    ELSE IF average >= 50, result = Second Class
    ELSE result = Pass
    DISPLAY total, average, result
END IF
STOP
```

Examination Result - Flowchart



For repeated items / days / transactions, repeat the input and process steps as required.

Examination Result - Source Code and Test Cases

Python Source Code

```
subjects = ["English", "Maths", "Science"]
marks = []

for subject in subjects:
    score = float(input(subject + " marks: "))
    marks.append(score)

if any(score < 0 or score > 100 for score in marks):
    print("Marks must be between 0 and 100")
else:
    average = sum(marks) / len(marks)

    if any(score < 35 for score in marks):
        result = "Fail"
    elif average >= 75:
        result = "Distinction"
    elif average >= 60:
        result = "First Class"
    elif average >= 50:
        result = "Second Class"
    else:
        result = "Pass"

    print("Total marks:", sum(marks), "out of 300")
    print("Average:", round(average, 2), "%")
    print("Result:", result)
```

Test Cases and Program Output

Input: 100, 100, 100
Expected output: Total 300; average 100%; Distinction

Input: 30, 80, 80
Expected output: Fail because one mark is below 35

Conclusion

The program checks each subject and then displays the result based on the pass rule and average.

Attendance Recovery

Problem Statement

Calculate current attendance and find how many consecutive classes are needed to reach the required percentage.

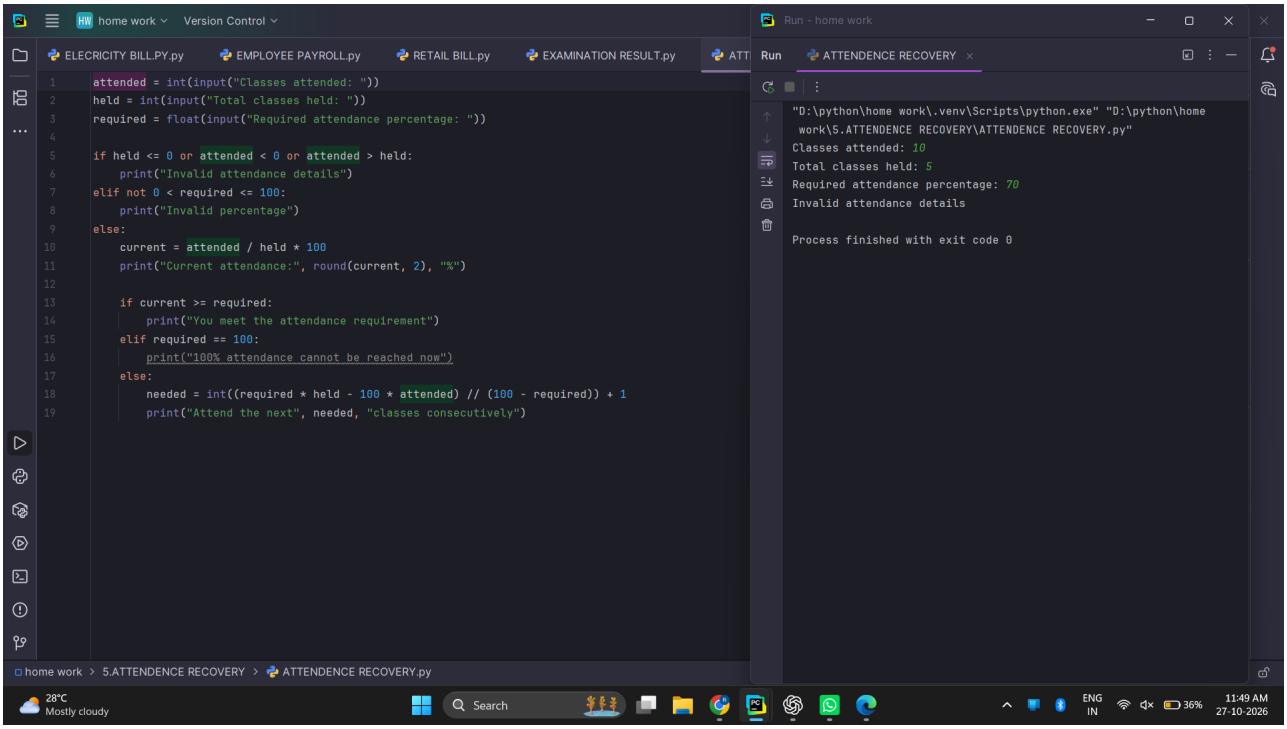
Objectives

To use input validation, percentages, conditions, and arithmetic.

Assumptions

Only future classes that are all attended are counted. If the required percentage is 100 and a past class was missed, the program reports that 100% cannot be reached.

Program Screenshot



Attendance Recovery - Algorithm and Pseudocode

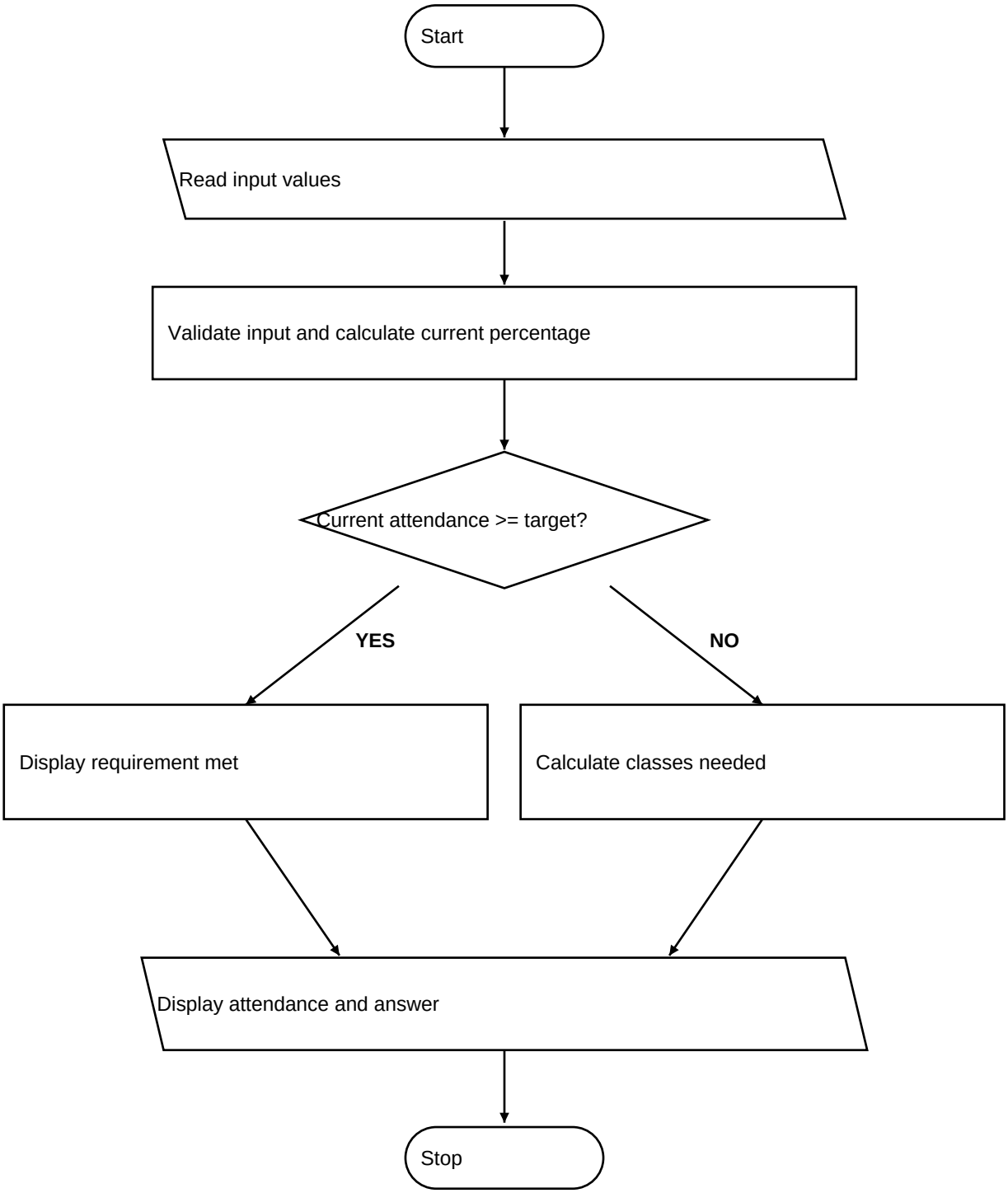
Algorithm

1. Read attended classes, classes held, and required percentage.
2. Check that the numbers are valid.
3. Calculate the current attendance percentage.
4. If the target is already met, display that it is met.
5. If the target is 100%, display the special message.
6. Otherwise calculate the number of classes to attend in a row.
7. Display the result.

Pseudocode

```
START
INPUT attended, held, required
IF held <= 0 OR attended < 0 OR attended > held
    DISPLAY invalid attendance details
ELSE IF required <= 0 OR required > 100
    DISPLAY invalid percentage
ELSE
    current = attended / held * 100
    IF current >= required, DISPLAY requirement met
    ELSE IF required = 100, DISPLAY cannot reach 100%
    ELSE calculate consecutive classes needed
    DISPLAY current attendance and answer
END IF
STOP
```

Attendance Recovery - Flowchart



For repeated items / days / transactions, repeat the input and process steps as required.

Attendance Recovery - Source Code and Test Cases

Python Source Code

```
attended = int(input("Classes attended: "))
held = int(input("Total classes held: "))
required = float(input("Required attendance percentage: "))

if held <= 0 or attended < 0 or attended > held:
    print("Invalid attendance details")
elif not 0 < required <= 100:
    print("Invalid percentage")
else:
    current = attended / held * 100
    print("Current attendance:", round(current, 2), "%")

    if current >= required:
        print("You meet the attendance requirement")
    elif required == 100:
        print("100% attendance cannot be reached now")
    else:
        needed = int((required * held - 100 * attended) // (100 - required)) + 1
        print("Attend the next", needed, "classes consecutively")
```

Test Cases and Program Output

Input: 10 attended, 5 held, target 70%
Expected output: Invalid attendance details

Input: 10 attended, 20 held, target 70%
Expected output: Current 50%; attend next 14 classes

Conclusion

The program checks the attendance data and tells the student whether the target is met or how many classes are needed.

ATM Transaction

Problem Statement

Simulate deposits and withdrawals, update the balance, and display transaction totals when the user finishes.

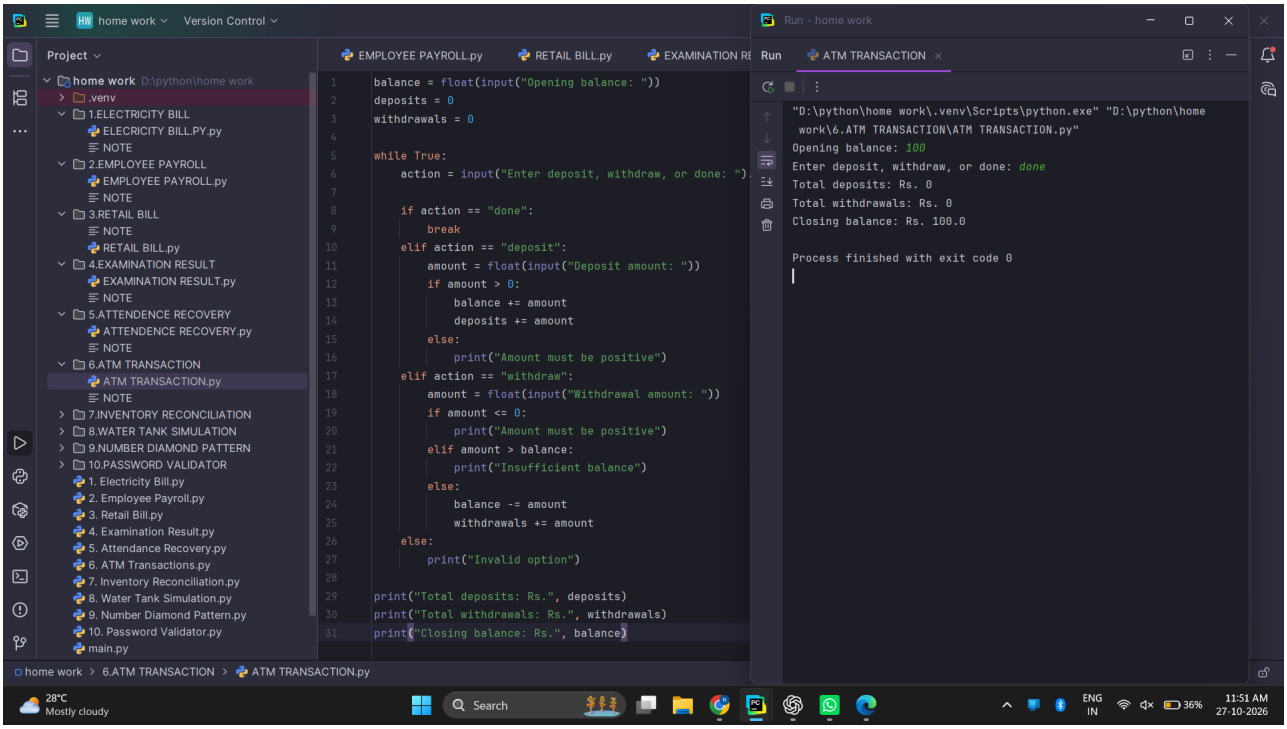
Objectives

To use a while loop, conditions, arithmetic, and input/output.

Assumptions

This is a practice simulation. Deposits and withdrawals must be positive. A withdrawal cannot be more than the current balance.

Program Screenshot



ATM Transaction - Algorithm and Pseudocode

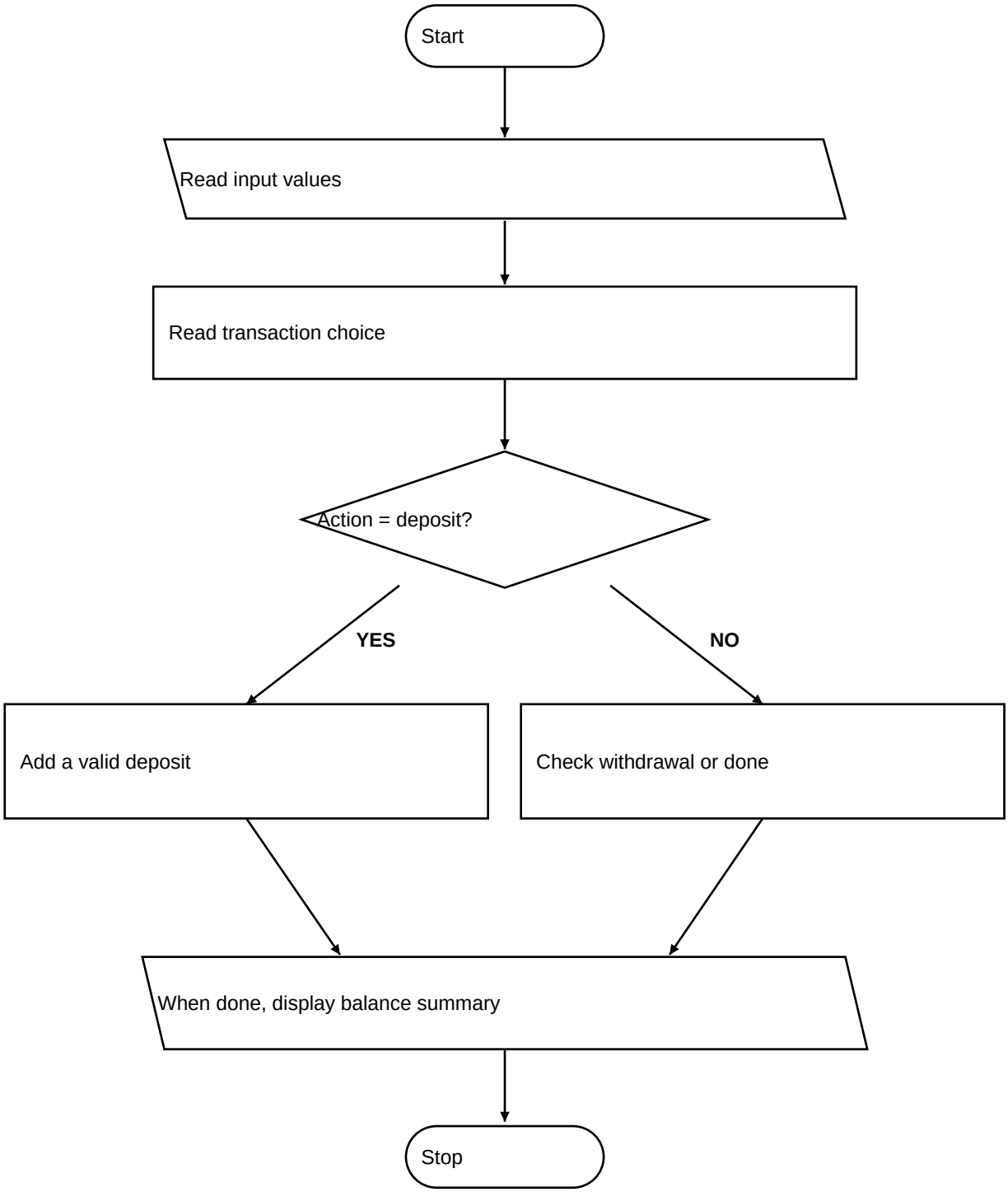
Algorithm

1. Read the opening balance and set deposit and withdrawal totals to zero.
2. Ask for deposit, withdraw, or done.
3. If done, stop taking transactions.
4. For a deposit, check the amount and add it to the balance.
5. For a withdrawal, check the amount and available balance.
6. Repeat the menu until done.
7. Display totals and closing balance.

Pseudocode

```
START
INPUT opening_balance
deposits = 0; withdrawals = 0
WHILE true
    INPUT action
    IF action = done, EXIT LOOP
    IF action = deposit, read amount and add if positive
    ELSE IF action = withdraw
        read amount
        IF amount <= 0, display error
        ELSE IF amount > balance, display insufficient balance
        ELSE subtract amount and add to withdrawals
END WHILE
DISPLAY deposits, withdrawals, closing balance
STOP
```

ATM Transaction - Flowchart



For repeated items / days / transactions, repeat the input and process steps as required.

ATM Transaction - Source Code and Test Cases

Python Source Code

```
balance = float(input("Opening balance: "))
deposits = 0
withdrawals = 0

while True:
    action = input("Enter deposit, withdraw, or done: ").lower()

    if action == "done":
        break
    elif action == "deposit":
        amount = float(input("Deposit amount: "))
        if amount > 0:
            balance += amount
            deposits += amount
        else:
            print("Amount must be positive")
    elif action == "withdraw":
        amount = float(input("Withdrawal amount: "))
        if amount <= 0:
            print("Amount must be positive")
        elif amount > balance:
            print("Insufficient balance")
        else:
            balance -= amount
            withdrawals += amount
    else:
        print("Invalid option")

print("Total deposits: Rs.", deposits)
print("Total withdrawals: Rs.", withdrawals)
print("Closing balance: Rs.", balance)
```

Test Cases and Program Output

Input: Opening Rs. 100; enter done
Expected output: Deposits 0; withdrawals 0; closing balance Rs. 100

Input: Opening Rs. 100; withdraw Rs. 150
Expected output: Withdrawal declined: insufficient balance

Conclusion

The program processes each valid transaction and keeps the account balance updated.

Inventory Reconciliation

Problem Statement

Compare recorded closing stock with the stock expected from opening stock, receipts, and dispatches.

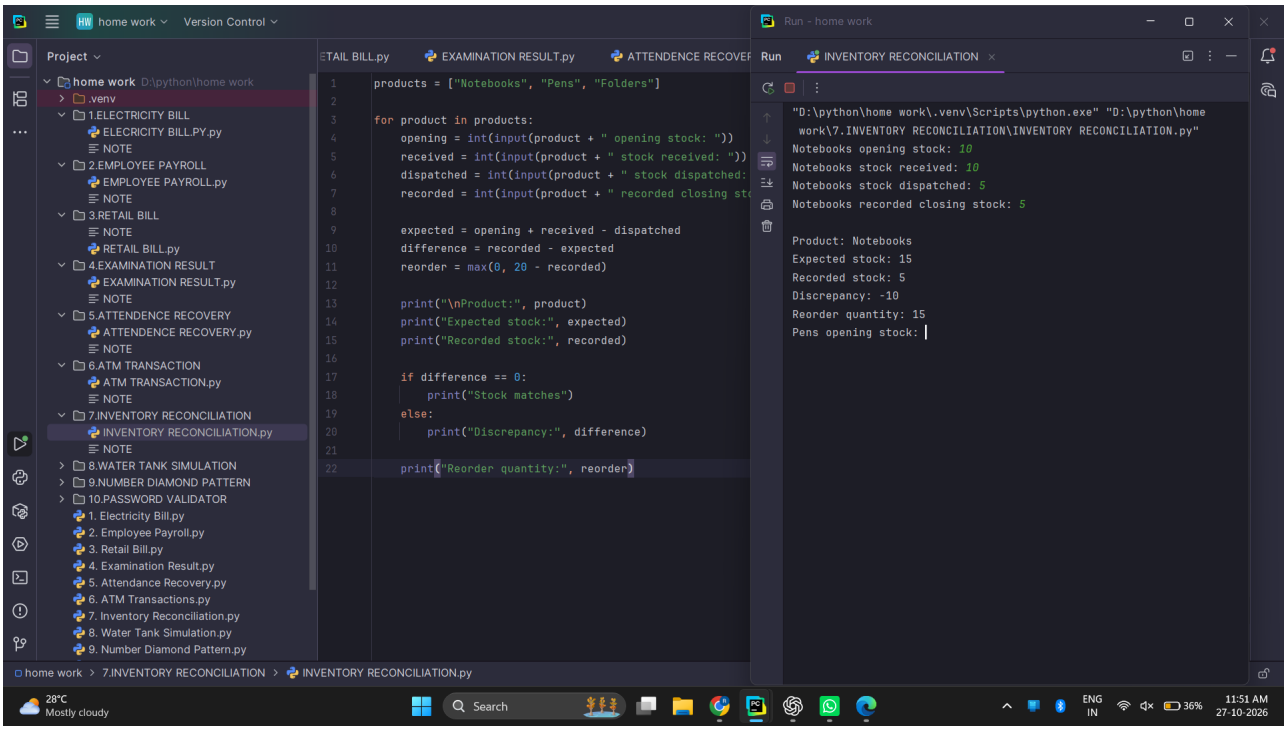
Objectives

To use a list, a loop, arithmetic, conditions, and stock comparison.

Assumptions

The product list contains Notebooks, Pens, and Folders. The desired minimum stock is 20 units.

Program Screenshot



Inventory Reconciliation - Algorithm and Pseudocode

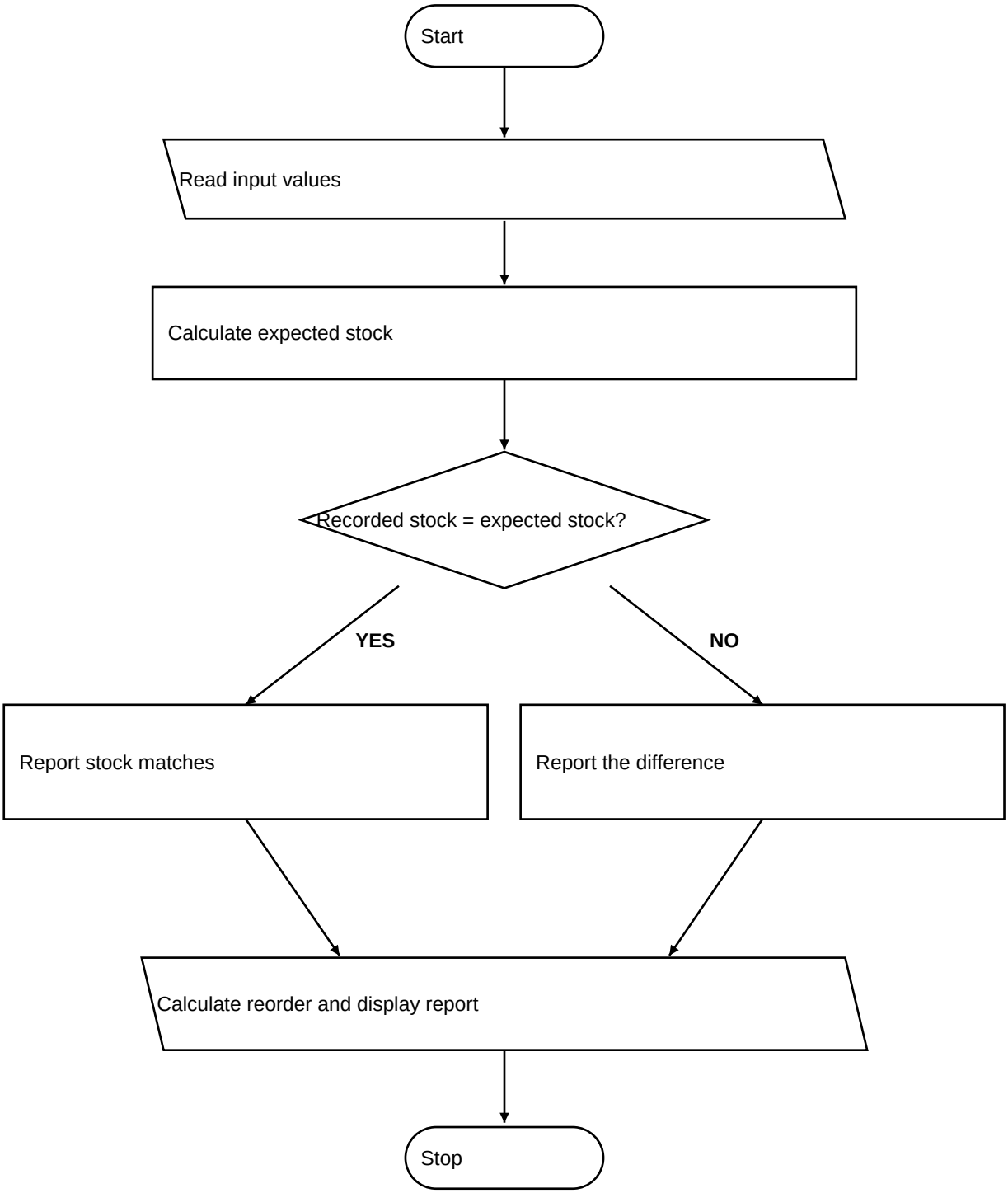
Algorithm

1. For each product, read opening, received, dispatched, and recorded stock.
2. Calculate expected closing stock.
3. Find the difference between recorded and expected stock.
4. Check whether the stock matches.
5. Calculate how many units are needed to reach 20.
6. Display the stock report.

Pseudocode

```
START
products = Notebooks, Pens, Folders
FOR each product
    INPUT opening, received, dispatched, recorded
    expected = opening + received - dispatched
    difference = recorded - expected
    reorder = maximum(0, 20 - recorded)
    IF difference = 0, DISPLAY stock matches
    ELSE DISPLAY discrepancy
    DISPLAY expected stock and reorder
END FOR
STOP
```

Inventory Reconciliation - Flowchart



For repeated items / days / transactions, repeat the input and process steps as required.

Inventory Reconciliation - Source Code and Test Cases

Python Source Code

```
products = ["Notebooks", "Pens", "Folders"]

for product in products:
    opening = int(input(product + " opening stock: "))
    received = int(input(product + " stock received: "))
    dispatched = int(input(product + " stock dispatched: "))
    recorded = int(input(product + " recorded closing stock: "))

    expected = opening + received - dispatched
    difference = recorded - expected
    reorder = max(0, 20 - recorded)

    print("\nProduct:", product)
    print("Expected stock:", expected)
    print("Recorded stock:", recorded)

    if difference == 0:
        print("Stock matches")
    else:
        print("Discrepancy:", difference)

    print("Reorder quantity:", reorder)
```

Test Cases and Program Output

Input: Notebooks: opening 10, received 10, dispatched 5, recorded 5
Expected output: Expected 15; discrepancy -10; reorder 15

Input: Recorded stock 20
Expected output: Reorder quantity 0

Conclusion

The program finds stock differences and shows the quantity needed to reach the chosen minimum.

Water Tank Simulation

Problem Statement

Update a tank's water level over several days using daily inflow and outflow.

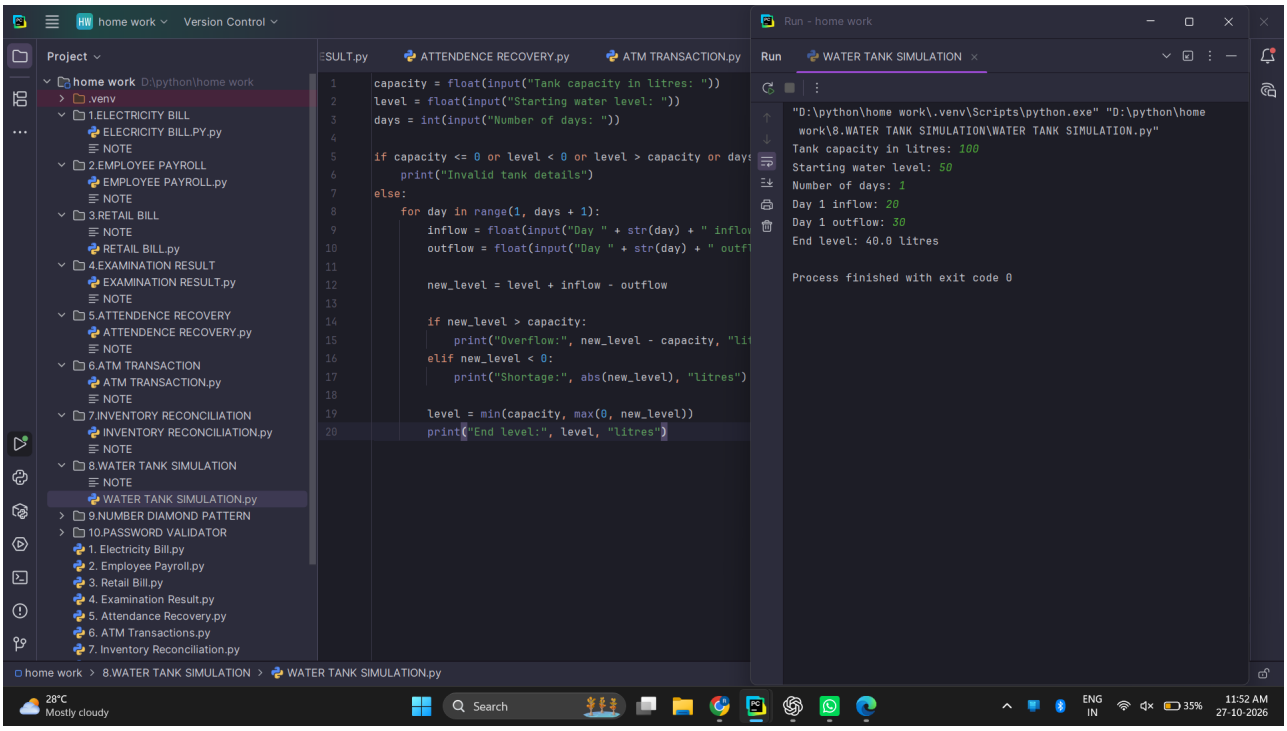
Objectives

To use a loop, arithmetic, conditions, and limits on a value.

Assumptions

The water level cannot be below zero or above tank capacity after each day. Water is measured in litres.

Program Screenshot



Water Tank Simulation - Algorithm and Pseudocode

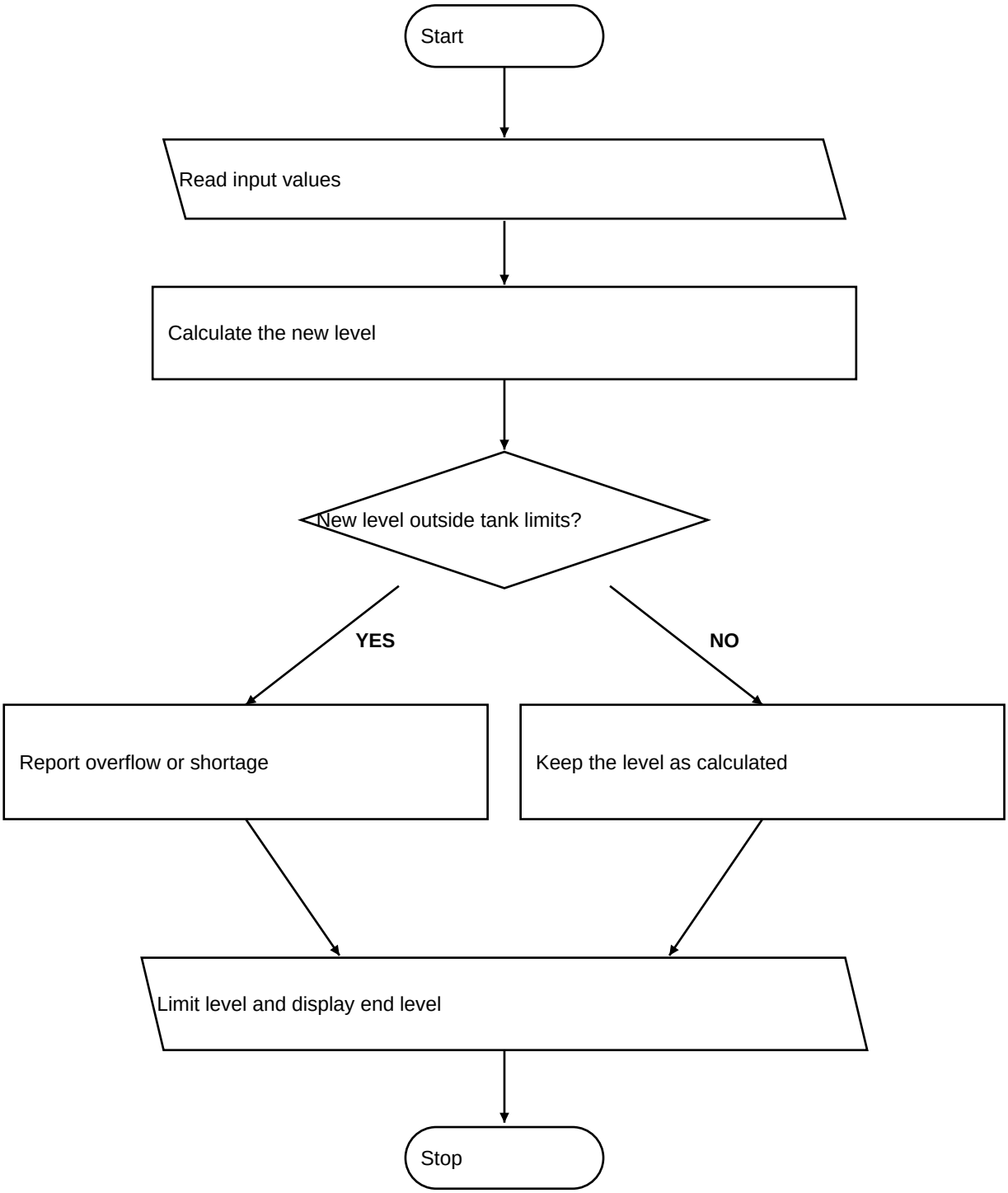
Algorithm

1. Read tank capacity, starting level, and number of days.
2. Check the tank details.
3. For each day, read inflow and outflow.
4. Calculate the new level.
5. Report overflow or shortage if needed.
6. Keep the stored level between zero and capacity.
7. Display the end level.

Pseudocode

```
START
INPUT capacity, level, days
IF tank details are invalid, DISPLAY error
ELSE FOR each day
    INPUT inflow, outflow
    new_level = level + inflow - outflow
    IF new_level > capacity, DISPLAY overflow
    ELSE IF new_level < 0, DISPLAY shortage
    level = minimum(capacity, maximum(0, new_level))
    DISPLAY level
END FOR
STOP
```

Water Tank Simulation - Flowchart



For repeated items / days / transactions, repeat the input and process steps as required.

Water Tank Simulation - Source Code and Test Cases

Python Source Code

```
capacity = float(input("Tank capacity in litres: "))
level = float(input("Starting water level: "))
days = int(input("Number of days: "))

if capacity <= 0 or level < 0 or level > capacity or days < 0:
    print("Invalid tank details")
else:
    for day in range(1, days + 1):
        inflow = float(input("Day " + str(day) + " inflow: "))
        outflow = float(input("Day " + str(day) + " outflow: "))

        new_level = level + inflow - outflow

        if new_level > capacity:
            print("Overflow:", new_level - capacity, "litres")
        elif new_level < 0:
            print("Shortage:", abs(new_level), "litres")

        level = min(capacity, max(0, new_level))
        print("End level:", level, "litres")
```

Test Cases and Program Output

Input: Capacity 100, start 50, inflow 20, outflow 30
Expected output: End level 40 litres

Input: Capacity 100, start 90, inflow 20, outflow 0
Expected output: Overflow 10 litres; end level 100 litres

Conclusion

The program tracks water each day and reports when the level goes outside the tank capacity.

Number Diamond Pattern

Problem Statement

Print a centered diamond pattern using number rows that grow and then shrink.

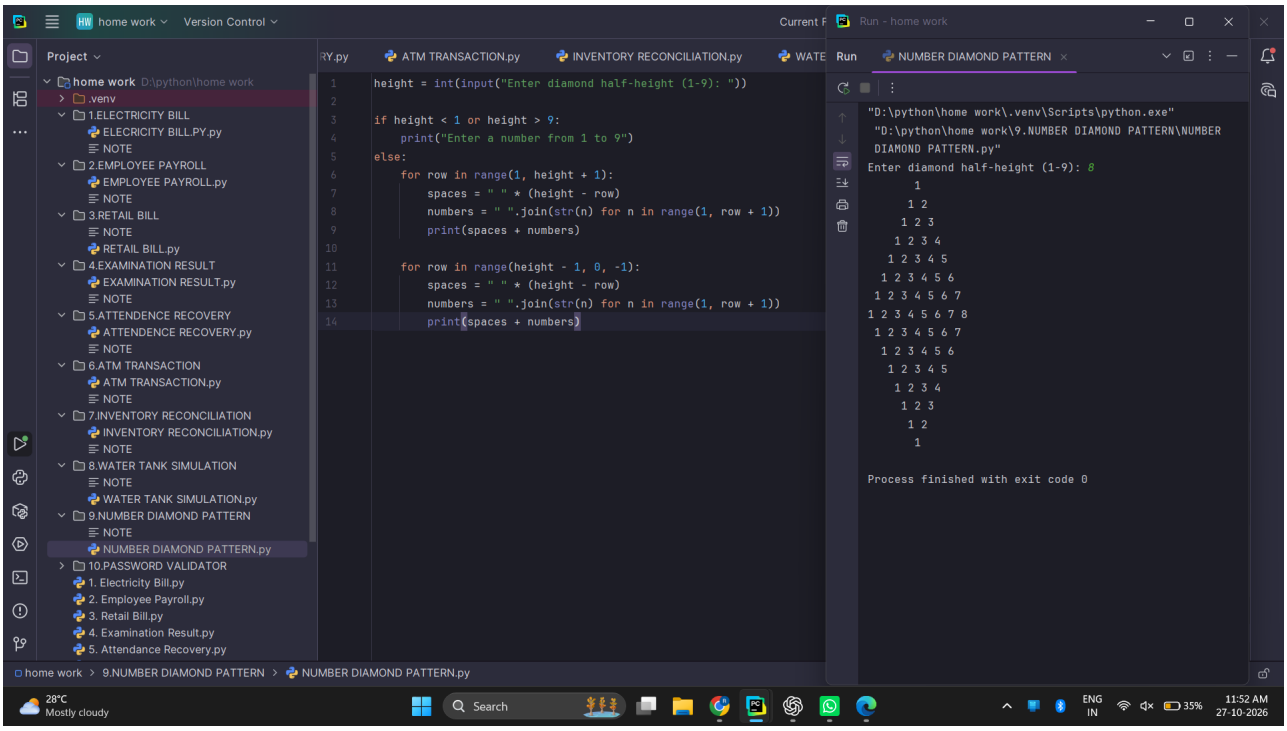
Objectives

To use nested loops, range, strings, and pattern generation.

Assumptions

The half-height must be an integer from 1 to 9. Spaces are added before each row to center the pattern.

Program Screenshot



Number Diamond Pattern - Algorithm and Pseudocode

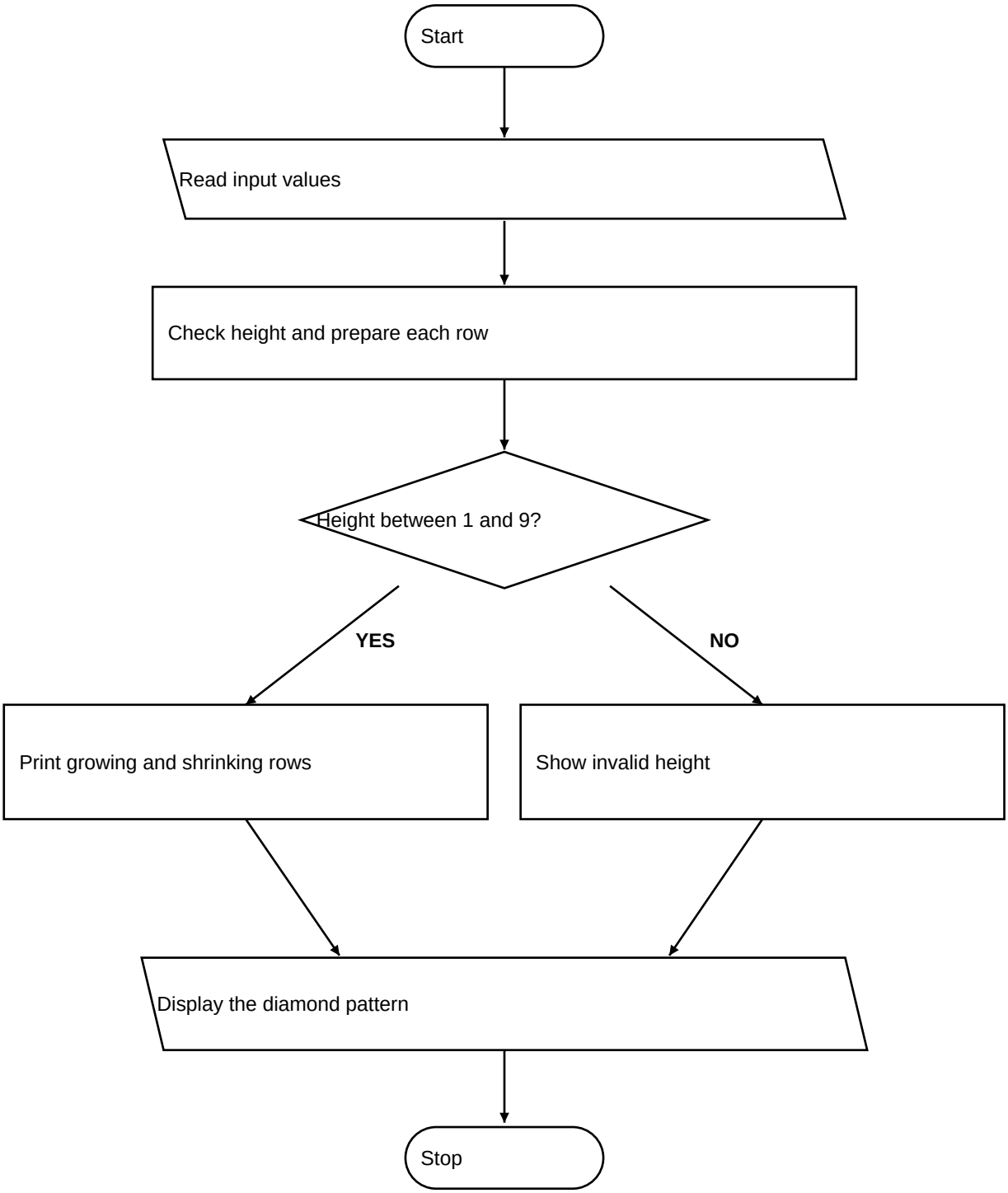
Algorithm

1. Read the diamond half-height.
2. If it is outside 1 to 9, display an error.
3. Print rows from 1 up to the height.
4. Print rows from height minus 1 back down to 1.
5. Stop after the diamond is printed.

Pseudocode

```
START
INPUT height
IF height < 1 OR height > 9
    DISPLAY invalid height
ELSE
    FOR row = 1 TO height
        Print height - row spaces and numbers 1 to row
    END FOR
    FOR row = height - 1 DOWN TO 1
        Print height - row spaces and numbers 1 to row
    END FOR
END IF
STOP
```

Number Diamond Pattern - Flowchart



For repeated items / days / transactions, repeat the input and process steps as required.

Number Diamond Pattern - Source Code and Test Cases

Python Source Code

```
height = int(input("Enter diamond half-height (1-9): "))

if height < 1 or height > 9:
    print("Enter a number from 1 to 9")
else:
    for row in range(1, height + 1):
        spaces = " " * (height - row)
        numbers = " ".join(str(n) for n in range(1, row + 1))
        print(spaces + numbers)

    for row in range(height - 1, 0, -1):
        spaces = " " * (height - row)
        numbers = " ".join(str(n) for n in range(1, row + 1))
        print(spaces + numbers)
```

Test Cases and Program Output

Input: Height 3
Expected output: Rows: 1 / 1 2 / 1 2 3 / 1 2 / 1

Input: Height 0
Expected output: Enter a number from 1 to 9

Conclusion

The two loops print the upper and lower halves of the number diamond.

Password Validator

Problem Statement

Check whether a password meets the length and character rules in the program.

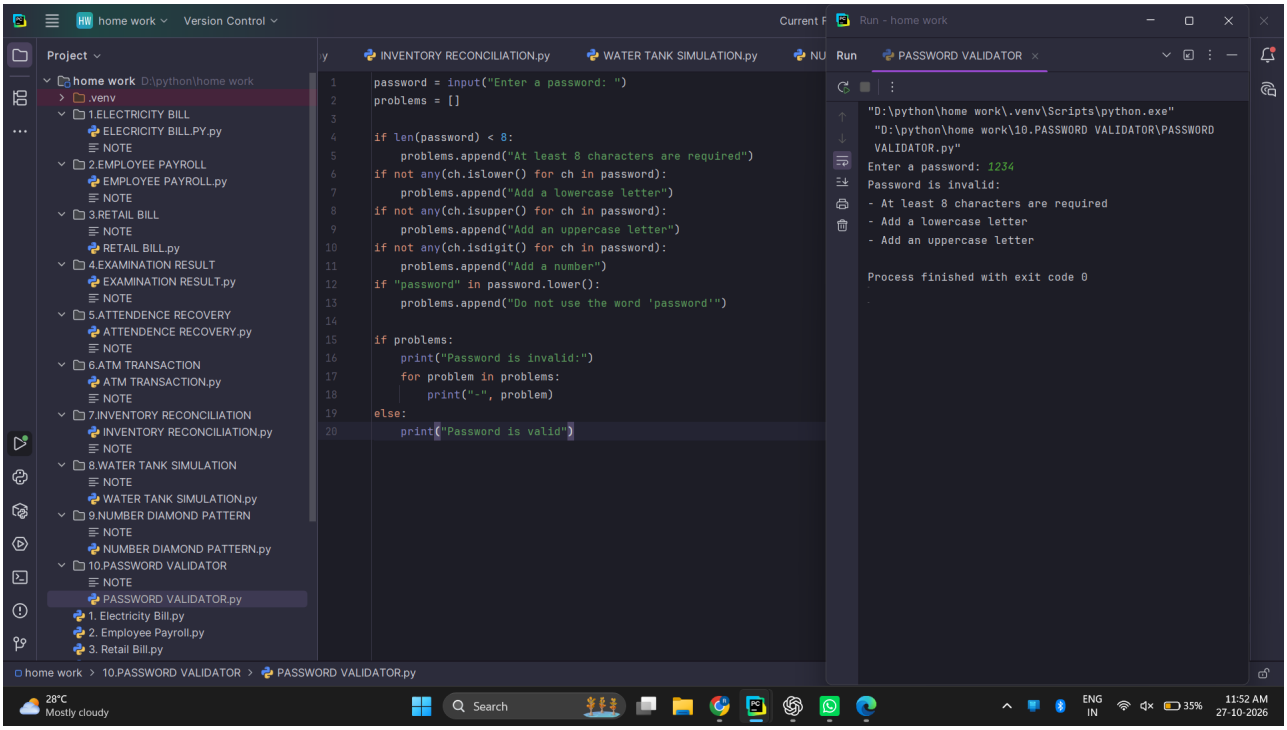
Objectives

To use strings, lists, loops, conditions, and character checks.

Assumptions

A valid password has at least 8 characters, one lowercase letter, one uppercase letter, and one digit. It must not contain the word password.

Program Screenshot



Password Validator - Algorithm and Pseudocode

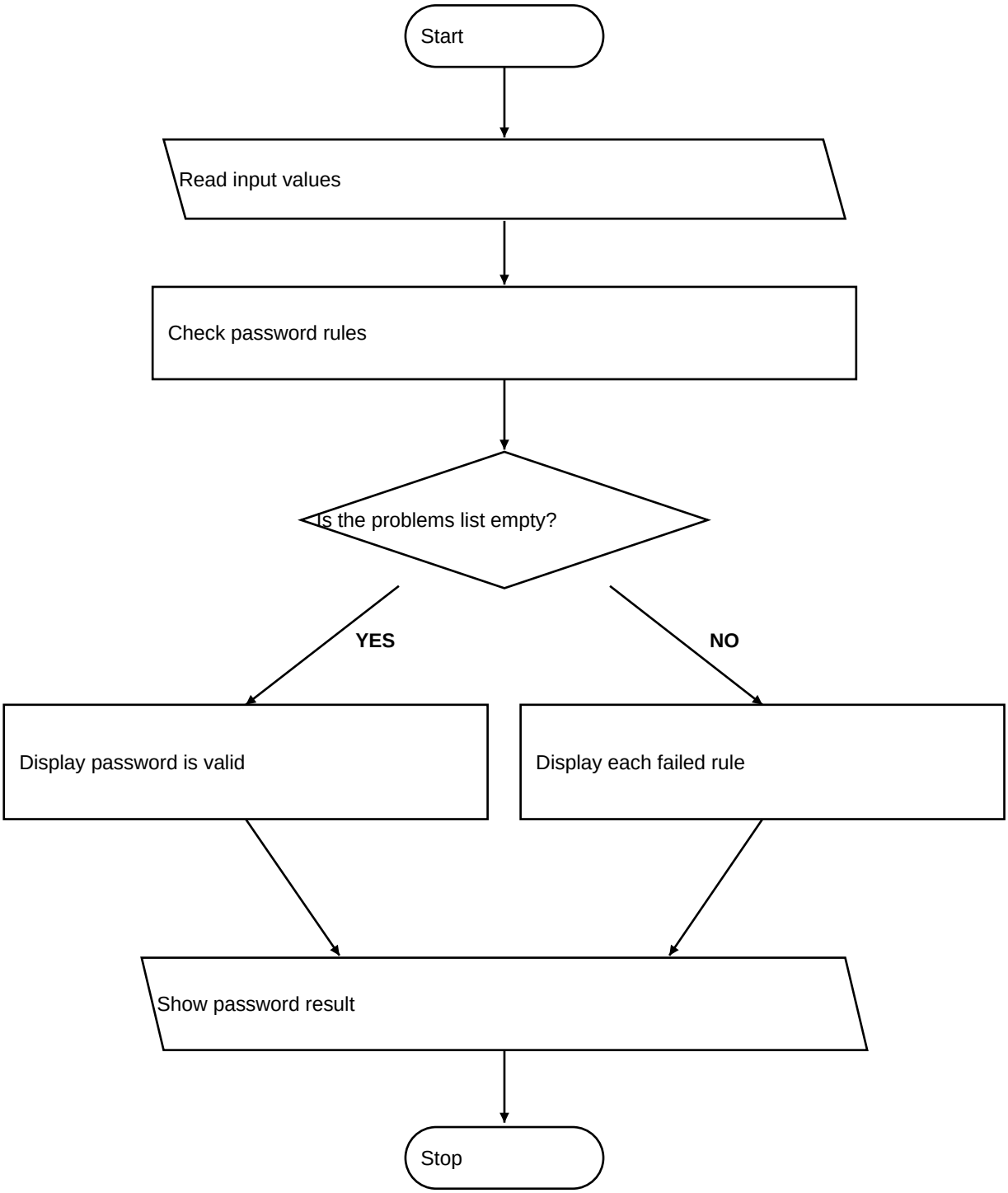
Algorithm

1. Read the password and create an empty list of problems.
2. Check its length.
3. Check for lowercase, uppercase, and digit characters.
4. Check that it does not contain the word password.
5. Add a message for each rule that fails.
6. Display valid or list the problems.

Pseudocode

```
START
INPUT password
problems = empty list
IF length < 8, add length message
IF no lowercase, add lowercase message
IF no uppercase, add uppercase message
IF no digit, add digit message
IF password contains "password", add warning
IF problems is empty, DISPLAY valid
ELSE DISPLAY each problem
STOP
```

Password Validator - Flowchart



For repeated items / days / transactions, repeat the input and process steps as required.

Password Validator - Source Code and Test Cases

Python Source Code

```
password = input("Enter a password: ")
problems = []

if len(password) < 8:
    problems.append("At least 8 characters are required")
if not any(ch.islower() for ch in password):
    problems.append("Add a lowercase letter")
if not any(ch.isupper() for ch in password):
    problems.append("Add an uppercase letter")
if not any(ch.isdigit() for ch in password):
    problems.append("Add a number")
if "password" in password.lower():
    problems.append("Do not use the word 'password'")

if problems:
    print("Password is invalid:")
    for problem in problems:
        print("-", problem)
else:
    print("Password is valid")
```

Test Cases and Program Output

Input: Password 1234
Expected output: Invalid: too short; missing lowercase and uppercase letters

Input: Password Abc12345
Expected output: Invalid: contains the word password

Input: Password Study123
Expected output: Valid under these rules

Conclusion

The program reports each rule the password does not meet.